SEIS 616

AI-Driven Cloud Infrastructure

Aaron Gesmer
Research presentation

ACM · FSE 2024

BARO

Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection

Luan Pham, RMIT University, Australia
Huong Ha, RMIT University, Australia
Hongyu Zhang, Chongqing University, China

doi.org/10.1145/3 660 805

MOTIVATION

Why I chose this paper

01

Alarms are often late

Most tools assume we know exactly when a failure started. In real life we usually do not.

02

It is cheap to try

Simple math on monitoring data most teams already collect. No AI training needed.

03

It was tested on a big app

64 connected services, not just a small demo store.

04

Anyone can check it

The code and data are public, and the results were independently verified.

THE PROBLEM

Why today's tools struggle

Simple alarms

They cry wolf

They catch real failures, but they also go off when nothing is wrong.

Map-based tools

Too big to map

They first draw a map of which services talk to which. With 64 services, the map often never gets finished.

A shared weakness

They trust the clock

If the failure time is a little late, broken data gets counted as normal, and the answer goes wrong.

HOW IT WORKS

Two steps: when, then what

1

Read the gauges

Speed, errors, CPU, and memory for every service.

2

Find when it broke

Watch speed and errors for the first moment things change.

3

Score every gauge

How far did each gauge move from its normal middle value?

4

Hand over a list

Gauges sorted by biggest move. Check the top first.

No training and no map needed. The first change tells you when it broke, not who broke it. The hallway smoke alarm goes off first, but the fire is in the kitchen.

STEP 1

Spot the moment things changed

Busy is not broken.

BARO watches the signals together and keeps asking: does the newest data still look like before? When the whole picture shifts at once, that is the start.

To decide, it only checks two things: is it slow, and is it throwing errors? Every other gauge is saved for step 2.

The Black Friday rule

Heavy traffic is fine if pages stay fast and error-free.

First change wins

That moment splits "normal" from "broken."

Nothing to tune

No alarm levels to set by hand for each gauge.

STEP 2

Rank the gauges by how far they moved

1 Learn normal

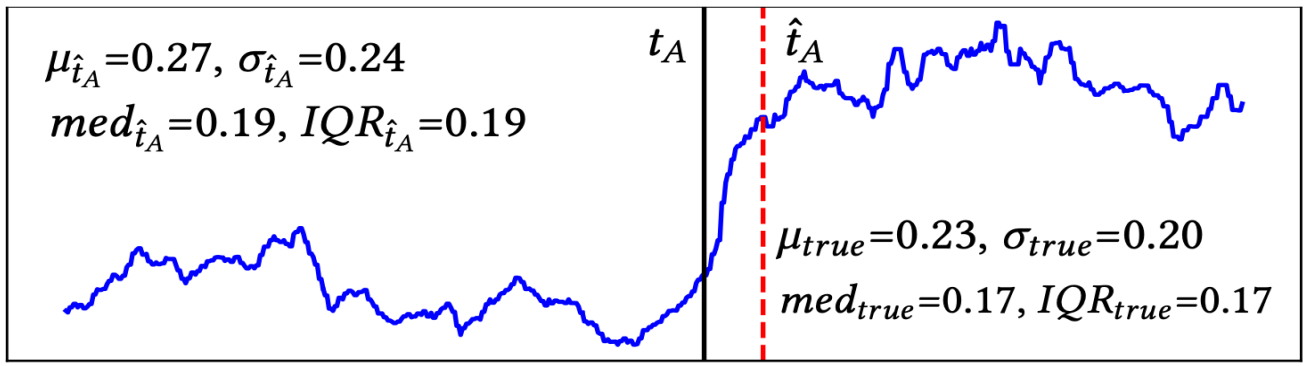
Before the break, find each gauge's middle value (the median).

2 Measure the move

After the break, how far did it move from that middle?

3 Rank them

Biggest move goes to the top. The top gauge points to the service.



Pham et al. (2024), Fig. 3b. Solid line: real start. Dashed red: start called late.

Why middle, not average

Call the start late and some broken data gets counted as normal. That drags the average. The middle barely moves.
Think of Bill Gates walking into a bar.

THE TEST

Three test apps, one big one

Online Boutique

12 services
49 gauges

A small online store.

Sock Shop

11 services
46 gauges

Another small store.

Train Ticket

64 services
212 gauges

Big. The real test.

100
tests per app

4
ways to break it

1–3 s
until symptoms show

5
servers

They broke things on purpose: overloaded the CPU, leaked memory, slowed the network, dropped data.
Each app was restarted fresh after every test.

RESULTS

The big app is where BARO pulls ahead

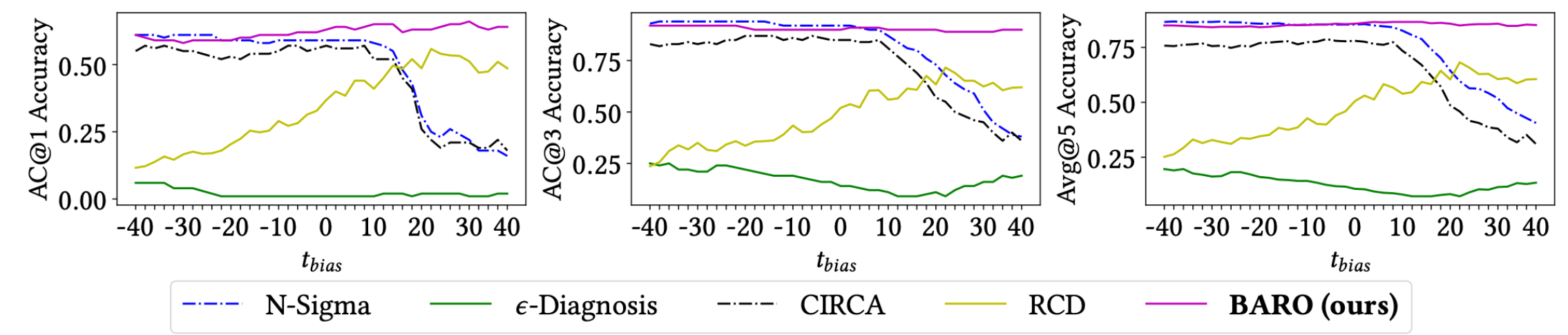
	Boutique	Sock Shop	Train Ticket
BARO alarm score	0.82	0.75	0.81
BARO culprit score	0.86	0.95	0.81
Map-based tool	0.80	0.60	**0.28**
Old simple tool	0.74	0.83	0.75

Culprit score: 1.0 means the real culprit is always first on the list. Higher is better.
On the small apps, everyone does fine. On the big app, the map-based tool drops to 0.28. BARO stays at 0.81.

BARO caught every failure. It still raised some false alarms, just fewer than the other tools.

RESULTS

Move the clock and BARO holds still



Pham et al. (2024), Fig. 5, small store app. Higher is better. Left of center: start called early. Right: called late.

Late is the danger

Call the start late and the average-based tools (N-Sigma, CIRCA) slide down.

BARO stays flat

Same tests, same shifts. BARO's pink line barely moves.

Swap only step 2

Keep BARO's step 1 but rank with averages: the big app drops from 0.81 to 0.30.

CAVEATS

What I would not overclaim

Close race

Naming the exact gauge

BARO is good at naming the broken service. Naming the exact gauge is a close race.

Perfect clock, tie

The edge is a guessed time

Give the old simple tool the exact failure time and it scores about the same as BARO.

15 of 100

Map tool on the big app

Its map-drawing step failed on 85 tests, and it got a guessed map, not a real one.

Lab conditions

Staged failures, one at a time

Four kinds of failure, caused on purpose, with a fresh restart each time.

REFLECTION

A to-do list, not a self-healing system

Ready now

Ranking takes about 0.01 seconds.
Nothing to train, no map to maintain.
Still works when the clock is off.
Code and data are public.

Not yet

Spotting the failure takes 30 seconds to 7 minutes.
Only tested on staged failures in a lab.
Never run for a week without a restart.

What I would add: logs and traces (the authors plan this too), a flag when several services broke together, and a test on a system that is not reset after every failure.

THE CLAIM I WOULD DEFEND

Watching the signals together means fewer false alarms. Using the middle value keeps the list right when the clock is off.

That is what still works when the system gets big.

Pham, L., Ha, H., & Zhang, H. (2024). BARO: Robust root cause analysis for microservices via multivariate Bayesian online change point detection. Proceedings of the ACM on Software Engineering, 1(FSE), Article 98.

https://doi.org/10.1145/3 660 805
github.com/phamquiluan/baro

Figures on slides 6 and 9 are from Pham et al. (2024), Figs. 3b and 5.